

ColdTrack AI: Evaluation Artifact

Integration, Iteration and Evaluation (final)

AIC COMPFEST 18 Final · Smart Logistics · 26 September 2026 · code version: **main @ c5785c1** plus the eval suite (a9a56fa); dashboard tested at frontend-ui @ 8e374a0

What is this document?

ColdTrack AI watches the sensors of a refrigerated truck and warns the operator before the cargo gets too warm. This document is the story of one working day, told with measurements. In the morning we measured the prototype from the qualifying round (the **baseline**) and found where it fails. We then rebuilt the core (**iteration 1**), changed four rules after looking at the new failures (**iteration 2**), connected the dashboard to the rebuilt system (**integration**), and finally tested the whole thing, screen included (**evaluation**). Everything below is measured by the test suite in the `eval/` folder; where we did not measure something, we say so.

Key terms

Term	Meaning
Breach	The cargo temperature goes outside its safe range (for example above 8 °C for vaccines).
Urgent case	A one-minute snapshot of a truck whose cargo really breaches within 30 minutes. This is when the operator must act.
Countdown	Minutes left before a breach (time-to-breach).
Status	AMAN = safe (green), WASPADA = warning (amber), KRITIS = critical (red).
Snapshot / trip	A snapshot is the state of one truck at one minute, with its history. A trip is one whole journey; 106 trips make the main test set.

Summary

The most dangerous failure is fixed. When a breach was less than 30 minutes away, the baseline showed green in 32% of cases. The final system shows green in 2%, and the whole dashboard shows the same status as the server in 4/4 demo scenarios (integration check). **But the fix has costs and several goals are still missed:** it passes 7 of 19 checks (baseline: 8 of 23, an older list). The countdown appears in only 23% of urgent cases (goal 80%), healthy trucks get a warning in 19% of snapshots (goal 10% or less), and the temperature forecast is 1.41 °C off on average, worse than the baseline's 0.20 °C and worse than guessing “no change” (0.19 °C).

What we measure (server, 106 test trips)	Baseline	Iteration 1	Iteration 2 = final	Goal
Urgent cases shown as green	32%	0%	2%	5% or less
Urgent cases shown as critical	25%	0%	16%	no goal set
Urgent cases with a countdown	62%	0%	23%	80% or more
Healthy snapshots with a warning or critical status	6%	33%	19%	10% or less
Sensor frozen during a real breach: flagged	10%	0%	37%	50% or more
Urgent cases treated as safe with 0.3 °C sensor noise	82%	50%	3%	50% or less
Forecast error at 30 minutes (°C)	0.20	1.41	1.41	below 0.8
Response time, 95th percentile (ms)	15	112	120	below 300
Checks passed	8 of 23	7 of 19	7 of 19	lists differ

“Urgent treated as safe with noise”: before, this meant the classifier said healthy; after, the status is green. Fault names before came from 7 classes, after from 3 events, so those rows are not compared.

1. How we tested

Idea: test what the **user really sees** (status, countdown, fault name, and finally the screen), not only the models inside. The pass marks are our own; we set them before iterating and kept them, so the numbers stay comparable across versions.

Test	What we did	What it can NOT tell us
Forecast	Compared the predicted cargo temperature 15/30/60 min ahead with what happened in 16,336 snapshots, and against a "no change" guess.	Most snapshots are calm, so an average hides the moments that matter.
Fault naming	Checked whether the three events (door open 20+ min, sudden outside heat, faulty sensor) are named when they are true. Repeated on random samples of trips to see how uncertain the numbers are.	Modes like cooling degradation have no event; they are judged only through status and countdown.
Urgent-breach warnings	Took only snapshots where the cargo really breaches within 30 minutes: what status was shown, was a countdown shown, how accurate was it?	Ground truth comes from simulators, not real trucks.
Stress tests	Added sensor noise (0.1 / 0.3 / 0.5 °C), shifted the clock by 7 hours, froze the temperature sensor during a real warming.	The noise is artificial: it shows sensitivity, not the exact error on a real truck.
Whole-server check	Sent 1,634 requests to the real API and compared with batch scoring (agreement: 100%).	Says nothing about the screen.
New simulator trips	Scored 225 held-out trips from the new physics simulator (never trained on) to make sure rule changes were not tuned only to the old test trips.	dataset_v3 is not in git (80 MB); the results are committed, the run cannot be repeated without the file.
Integration check (new)	A script drives the real dashboard in a browser against the real server: 4 demo scenarios, 3 CSV cases (good, too short, wrong columns), 3 failure cases (server unreachable, server error, slow server).	One browser and one screen size; not run inside Docker; not a usability test with real operators.

Other checks that pass: backend tests 25 of 25; dashboard type-check and lint clean.

Two test sets, and why: the 106 **old-generator trips** (dataset v4) are the same ones the baseline was scored on, so all versions are comparable. But the rebuilt models were trained on a different simulator, so v4 is out-of-distribution for them, and some results may reflect that gap and not the truck. The 225 **new-simulator trips** (dataset v3) are the fair test for the rebuilt models.

2. Baseline: where the qualifying-round prototype fails (tag checkpoint-1-baseline)

The baseline passed 8 of 23 checks. Model scores from the qualifying round reproduced exactly (fault score 0.58, forecast error 0.20 °C), but the server often failed the operator. Full detail: [Evaluation_Artifact_baseline.pdf](#).

#	Finding	Evidence
F1	Urgent cases shown as green.	AMAN 32%, WASPADA 43%, KRITIS 25% (n=312).
F2	Countdown hidden when the classifier says "healthy".	68% of urgent cases were judged healthy by the classifier; the models alone showed a countdown in 25%.
F3	Sensor noise confuses it.	With 0.3 °C noise, urgent cases judged healthy rose from 68% to 82%.
F4	A frozen sensor is read as healthy.	73% predicted healthy; 10% flagged as sensor problem.
F5	The forecast target is met by a trivial guess.	"No change" scores 0.195 °C against the model's 0.198 °C.
F6	Three fault types mostly missed.	Found: outside heat 44%, poor pre-cooling 20%, weakening cooling 11%.
F7	Test data too thin for per-class or per-cargo claims.	2 to 4 test trips for several fault types; no urgent cases for vaccine, frozen meat or fish.

Cause we found: the status followed a forecast that was roughly "no change", and the countdown was hidden whenever the classifier believed the truck healthy. The classifier was weakest on exactly the slow faults that end in a breach.

3. Iterations: what we changed and what happened

Iteration 1 (tag checkpoint-2-iteration)

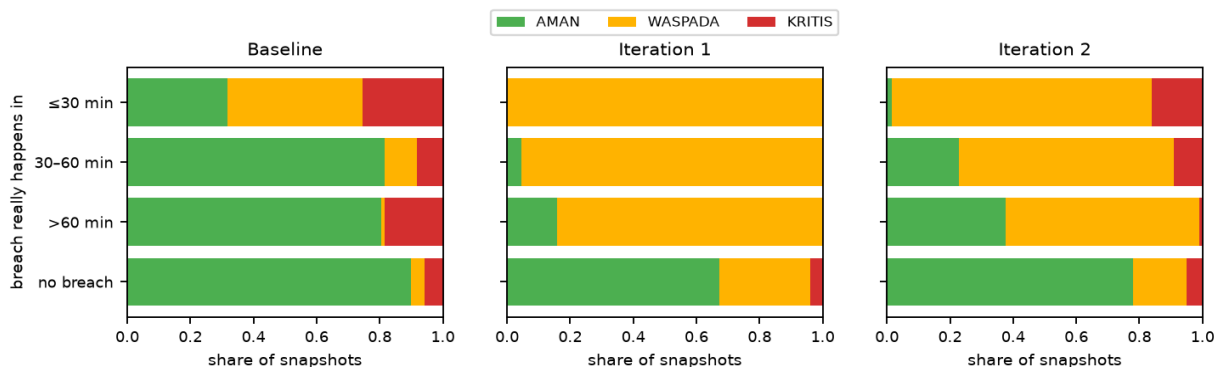
We replaced the prototype with a **physics engine** that estimates the cargo temperature from the cabin sensor, counts down to a breach and sets the status; a set of **sensor rules**; and a **learned model** (GRU) that names three events and forecasts the cargo temperature (an XGBoost countdown is kept as information only). The countdown no longer depends on the fault classifier, which was the baseline's main failure. The models were trained on a new physics simulator with real Indonesian climate data (BMKG) and real faulty-sensor patterns (Intel Berkeley Lab).

Result: urgent cases shown as green fell from 32% to 0%. But the cost was large: the countdown was never shown (0%), healthy trucks were warned in 33% of snapshots (baseline 6%), and the forecast got worse ($0.20 \rightarrow 1.41$ °C). Door open was never named (0%).

Iteration 2 (rule changes, no retraining)

Change	Finding it targets	New rule
Drift countdown	Urgent cases were warned but never given a countdown or critical status.	When the smoothed reading rises at least 0.3 °C per hour over 20 minutes and the door has been shut for 30 minutes, count the minutes until the reading itself would cross the limit; use the shorter of this and the physics countdown.
Margin scaled to cargo range	Healthy trucks warned in 33% of snapshots.	The “near the limit” warning starts at 35% of the cargo range, at most 1 °C (0.7 °C for 2–4 °C cargo).
Sensor reading as upper bound	Found while testing the margin change: the extreme-heat demo showed green with the sensor above the limit.	The margin rule uses the higher of the estimated cargo temperature and the smoothed sensor reading.
Stuck sensor after a rise	A sensor frozen 30 min before a breach was never flagged.	A flat reading becomes suspect once the trend of the 30 minutes before it would have moved it by 0.3 °C.

Status bars. Each bar groups snapshots by how soon the cargo really breaches; the top bar should be red.



Result on the 106 old trips: countdown shown in 23% of urgent cases (iteration 1: 0%, baseline 62%) and accurate when shown (off by 9.5 minutes); critical status for 16% of urgent cases (was 0%); frozen sensor flagged in 37% (was 0%); healthy warnings down from 33% to 19%; urgent cases treated as safe under 0.3 °C noise: 50% → 3%.

Baseline findings F1–F7: what we changed and what we did not

Same 106 test trips throughout; open items point to section 5.

#	Baseline finding	What we changed	Now	Verdict
F1	Urgent cases shown as green.	The physics engine sets the status; it no longer follows the forecast.	Green 32% → 2% (4% over all urgent snapshots). Most urgent cases are amber; critical only 16%.	Fixed
F2	Countdown hidden when the classifier says “healthy”.	Removed that dependence; the countdown comes from physics, plus a drift rule in iteration 2.	Countdown shown 62% → 23% (goal 80%). Physics cannot project a slow drift: 78% stay above the 60-minute display limit (N1).	Cause removed, outcome worse
F3	Sensor noise confuses it.	Status no longer depends on the classifier. Models not retrained with noise.	Urgent cases treated as safe at 0.3 °C noise 82% → 3%. Fault naming still collapses at 0.1 °C (mean event recall 27% → 5%) (N5).	Partly fixed
F4	A frozen sensor is read as healthy.	Added sensor rules; iteration 2 added “stuck after a rise”.	Flagged 10% → 37% (goal 50%); shown green 10%, was 73% judged healthy. Fires only if the reading was rising before it froze (N6).	Partly fixed
F5	Forecast target met by a “no change” guess.	Nothing: the model was not retrained. Only the reporting changed.	Forecast error 0.20 → 1.41 °C; “no change” scores 0.19 °C. The dashboard chart draws this forecast (N3).	Not fixed, regressed
F6	Three fault types mostly missed.	Scope narrowed to 3 events (door open, outside heat, faulty sensor). Cooling degradation, reefer off and poor pre-cooling are no longer named.	Door named 59% → 0%; outside heat 44% → 15%; faulty sensor 67% → 67%. Dropped types are judged only through status and countdown (N4).	Not fixed, scope cut
F7	Test data too thin.	Added a second test set: 225 held-out trips from the new simulator.	The old set is unchanged (2–4 trips for several fault types; no urgent cases for vaccine, frozen meat, fish). The new set is also simulated (N7).	Partly addressed

Trade-off we could not avoid: how wide should the “near the limit” margin be?

Margin for 2–4 °C cargo	Urgent shown green	Healthy warned
0.5 °C	7%	13%
0.6 °C	6%	16%
0.7 °C (chosen)	4%	20%
0.8 °C	0%	25%
1.0 °C	0%	33%

No width meets both goals on the old test trips: urgent and healthy snapshots sit at overlapping distances from the limit. 0.7 °C is the narrowest that keeps urgent-green near the 5% goal; 0.5 °C would cut healthy warnings to 13% but show 7% of urgent cases as green. The team chose safety.

Results on the new simulator's held-out trips (225 trips the models never trained on)

Per trip	Iteration 1	Iteration 2	Trips
Breaching trips warned before the breach	100%	100%	76
Breaching trips critical before the breach	92%	95%	76
Median lead of the critical status (min)	44	54	
Healthy trips with any warning	0%	0%	58
Door trips without a breach: any warning	81%	96%	27
Door trips without a breach: critical	48%	63%	27
Sensor-fault trips flagged	99%	99%	75

On the fair test set nothing got worse for healthy trucks or sensor faults and the critical status comes earlier (median 44 → 54 minutes before the breach). The price: 63% of trips where the door opened but the cargo never breached now reach critical (48% before). We do not know whether these are false alarms: it depends on where the probe sits

(section 5).

What went wrong and why (things that got worse, not only better)

What got worse	Why (measured unless marked)
Countdown coverage: 62% → 23%	The baseline showed a countdown from its models plus a fallback estimate in the server; the physics countdown only projects a breach when the cabin air is already above the limit. In the old test trips urgent cases are slow drifts, so 78% stay above the 60-minute display limit. Drift rule recovered part of it.
Healthy warnings: 6% → 19%	One rule (cargo within a margin of the limit) causes them all. Dairy and produce allow only 2–4 °C, and healthy cargo in the old trips runs inside that margin. The new simulator keeps the same cargo at its setpoint, so this may be partly a generator gap (hypothesis).
Forecast error: 0.20 → 1.41 °C	The model learned the new simulator's cooling setpoints, so it predicts frozen meat about 2 °C warmer than in the old trips. It was not retrained in iteration 2, so this is unchanged.
Door named: 59% → 0%	In the new simulator the cabin reading jumps 14–36 °C per minute when a door opens, a known flaw (no heat stored in walls and racks); in the old trips an open door barely moves the reading. The physics rules still raise the status for many door cases.

4. Integration: connecting the dashboard to the rebuilt system

What was connected. (1) The server (/api/v1/analyze) now runs the physics engine, sensor rules and models, and also serves the demo scenarios. (2) The new dashboard (pull request #25) builds a 60-minute window from the cargo form and shows status, countdown, truck picture, forecast chart, actions and drivers. (3) The header now has a scenario dropdown and an Import CSV button (commit 8e374a0, on branch frontend-ui, not yet on main at the time of writing): both send real telemetry to the same server instead of the form's synthetic window. The four dropdown entries A0, A-1, A-2 and A-3 map to the backend scenarios normal, door open, extreme ambient and stuck sensor. Tested combination: dashboard at commit 8e374a0, server code at commit a9a56fa (main is c5785c1).

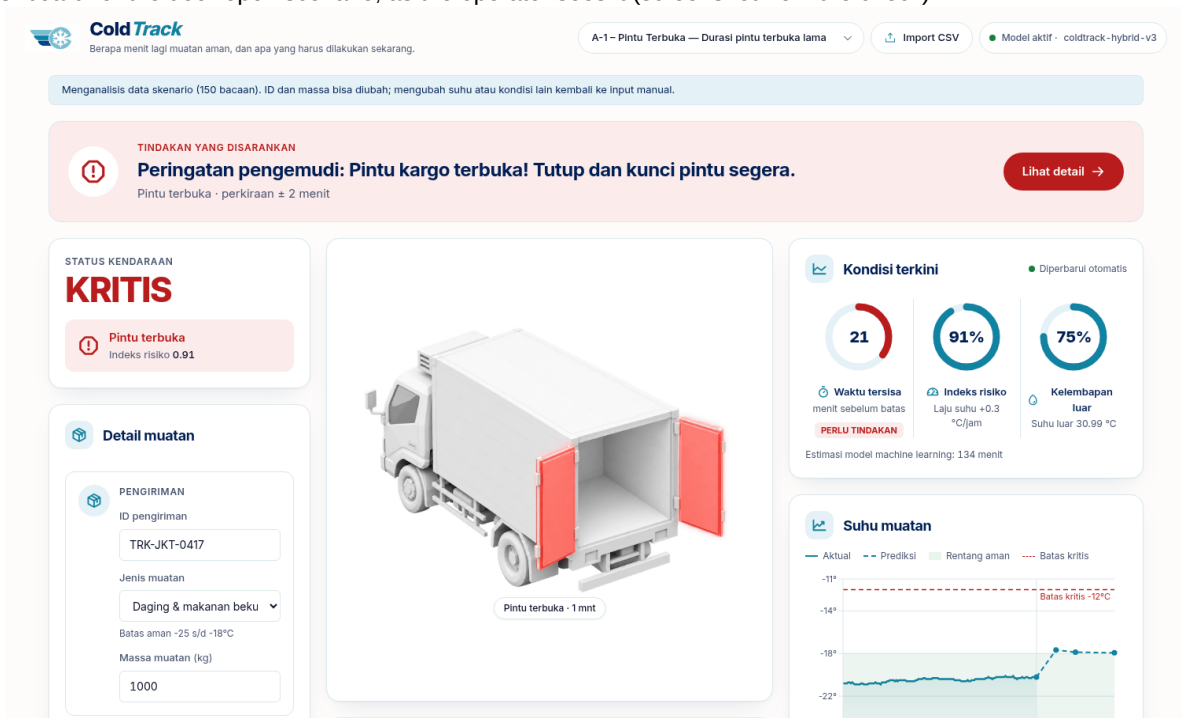
How we checked it. The script eval/ui_check.py opens the dashboard in a real browser against a real server and compares what the screen shows with what the API returns for the same data.

Dropdown	Expected status	Server says	Screen shows	Countdown	Fault name
A0 – Aman	AMAN	AMAN	AMAN	none	normal_sehat
A-1 – Pintu Terbuka	KRITIS	KRITIS	KRITIS	21 min	pintu_terbuka
A-2 – Kejut Ambient	WASPADA	WASPADA	WASPADA	none	suhu_muatan_mendekati_batas
A-3 – Sensor Macet	WASPADA	WASPADA	WASPADA	none	masalah_sensor

Screen equals server in 4/4 scenarios and the server meets its own scenario expectation in 4/4. No errors appeared in the browser console during normal use (0).

Case	What the user sees	Result
CSV, 90 rows, has temp_c	Result shown, button shows the file name	ok
CSV, 20 rows	CSV berisi 20 baris; model butuh minimal 60 bacaan (satu per menit).	ok
CSV without temp_c	CSV harus punya kolom `temp_c`. Kolom lain yang dikenali: ts, humidity, ambient_c, door_open, reefer_on, speed...	ok
Server unreachable	Server analisis tidak bisa dihubungi (http://localhost:8010). Pastikan backend berjalan. (with a “Coba lagi” retry button)	ok
Server error 500	Server mengembalikan error (500).	ok
Slow server	A loading state is shown while waiting	ok

Dashboard for the door-open scenario, as the operator sees it (screenshot from the check):



5. Findings not yet fixed, and why

Causes are marked *measured* or *hypothesis*. Plans are for after the hackathon; nothing below is claimed as done.

Finding	Where we stand	Cause and next step
N1 Countdown for urgent cases	Shown in 23% (goal 80%).	Physics cannot project a slow drift (measured). Next: fall back to the learned countdown (XGBoost) when neither physics nor drift can project; judge on both test sets.
N2 False warnings on healthy trucks	19% of snapshots (goal 10%).	The margin alone cannot separate slow approaches from healthy running (measured; see sweep). Needs an extra signal.
N3 Forecast	1.41 °C off; "no change" is 0.19 °C.	Learned the new simulator's setpoints (measured); the exact cause of the bias is a hypothesis. Next: predict the change, not the level. The dashboard chart shows this forecast, so the drawn prediction can be about 1.4 °C off; for 2–4 °C cargo that is most of the safe range.
N4 Door and outside heat named	Door 0%, outside heat 15%.	Simulator flaws (no wall/rack heat storage, no sunlight) (measured differences). Next: fix the simulator and retrain. The physics rules still raise the status in many door cases.
N5 Noise	Event names collapse at 0.1 °C noise; status holds.	Trained with 0.073 °C noise only (known). Next: train with 0.05–0.5 °C.
N6 Stuck sensor	Flagged in 37% (goal 50%); shown green in 10%.	The rule fires only when the reading was rising before it froze (measured). Next: flag a flat reading when the estimated cargo should be warming.
N7 Test validity	Two test sets, both simulated.	No real truck data exists in this project. Results are evidence about the simulators and the rules, not about the field.
Sensor position (open question)	Unanswered.	The engine assumes a probe in the cabin air; the API says temp_c is cargo temperature. If in the cargo, the new rules are exact; if in the air, they warn earlier than needed (the door-trip cost in section 3). Ask the hardware owner.
Integration limits	Not measured.	Header controls on branch frontend-ui, not yet on main at the time of writing. Not run inside Docker in this check; one browser and one screen size; no usability test with real operators; the compressor-failure demo scenario is hidden from the dropdown (that fault is postponed) but still exists in the server.

What we would tell a judge in one sentence: the system no longer tells an operator "safe" when a breach is near, the dashboard shows exactly what the server says and handles bad input and outages, but it still misses the countdown in most urgent cases, warns too often on healthy trucks, and its temperature forecast is not yet useful.

Appendix A: acceptance checks (final = iteration 2 engine)

Result	Check
PASS	test_recall_per_event_at_least_0_5[sensor]
PASS	test_imminent_breach_not_shown_as_safe
PASS	test_sensor_noise_0_3C_does_not_hide_imminent_breach
PASS	test_timezone_shift_does_not_change_event_recall_much
PASS	test_imminent_breach_never_green_more_than_5pct
PASS	test_api_no_http_errors_and_latency_p95_under_300ms
PASS	test_api_matches_batch_scoring
FAIL	test_forecast_mae_t30_under_0_8C
FAIL	test_forecast_worst_case_error_bounded
FAIL	test_recall_per_event_at_least_0_5[door]
FAIL	test_recall_per_event_at_least_0_5[shock]
FAIL	test_healthy_windows_not_labelled_as_fault
FAIL	test_imminent_breach_ttb_shown_at_least_80pct
FAIL	test_test_split_covers_every_cargo_profile_for_imminent_breach
FAIL	test_sensor_noise_0_3C_event_recall_drop_at_most_0_05
FAIL	test_stuck_sensor_during_breach_is_flagged_not_safe
FAIL	test_imminent_breach_ttb_shown_by_api_at_least_80pct
FAIL	test_healthy_windows_not_over_alerted
FAIL	test_every_fault_mode_has_at_least_5_test_trips

The final run (commit a9a56fa) reproduced every one of the 231 numeric results of the iteration 2 run, which was measured at c336a2a (response times excluded). Backend and ml code are identical between c336a2a and main c5785c1 (git diff is empty), so the iteration 2 numbers stand for main.

Appendix B: how to reproduce

```
python -m eval.run_eval --tag final (about 4 min) · EVAL_TAG=final pytest
eval/test_baseline.py · python -m eval.ui_check --ui http://localhost:3000 --api
http://localhost:8000 (needs pip install playwright and Chrome; frontend built with
NEXT_PUBLIC_API_URL) · python -m eval.make_final_artifact. The baseline cannot be re-run (its models
were removed); its results are archived in results/baseline_results.json.
```

Appendix C: what was committed when (26 September 2026, WIB)

Time	Commit	Author	What
09:46	6494775	Faza Dzil	test(eval): add baseline evaluation suite and Evaluation Artifact
10:24	a3196b2	Jordan	test(eval): rerun baseline evaluation suite and Evaluation Artifact
10:30	f351c03	Jordan	test(eval): change Evaluation Artifact
10:50	6e6a536	Faza Dzil	docs(eval): plain-language artifact sections
14:01	07c0229	Falzii	fix(model): simulator fisika v3, GRU, observer, door_budget, aturan sensor
14:09	640c84f	Falzii	docs(model): add model docs
14:20	724d695	Falzii	fix(model): engine
14:23	43b33f6	Falzii	fix(model): GRU, XGBoost TTB
14:27	84e07b6	Falzii	fix(model): update context md
14:42	433af50	Falzii	feat(model): run train XGBoost
14:52	648ba01	Jordan	feat(api): ganti backend ke model hibrida checkpoint-2
15:40	ff9aacd	Jordan	test(eval): update test suite and evaluation artifact to match new contract

Time	Commit	Author	What
16:26	fa0e42c	Jordan	chore: ignore generated simulator dataset parquet files
16:27	bcdc6a4	Jordan	fix(rules): detect stuck sensor preceded by rising temperature
16:28	48cdae7	Jordan	feat(engine): add drift TTB projection and refine alert margins
16:28	cfb196b	Jordan	test(eval): add sim_check regression script for simulator test split
16:29	62e776e	Jordan	test(eval): record iter2 evaluation results
16:38	c336a2a	Jordan	test(eval): iteration-2
18:13	55a045f	Tsabita Khoirunnisa	feat(ui): dashboard input manual dengan gaya mockup
18:50	8e374a0	Faza Dzil	feat(ui): add scenario dropdown and CSV import to header
19:04	a9a56fa	Faza Dzil	test(eval): final artifact for integration, iteration and evaluation
19:12	60cc4d7	Faza Dzil	test(eval): re-measure final artifact on latest main
19:21	fd8df5f	Faza Dzil	docs(eval): add F1-F7 changed/not-changed table to final artifact
19:22	446e685	Faza Dzil	docs(eval): rebuild final artifact PDF

Tags at the time of writing: checkpoint-1-baseline → e846165 11:28; checkpoint-2-iteration → a136d2d 15:00; checkpoint-2-iteration-2 → 564f2ea 15:43; checkpoint-3-integration → 564f2ea 15:43. The last two tags point at the same commit, before the iteration 2 rule changes and before the dashboard merge; a tag on the final commit is still to be created.